

Relazione tecnica per il corso di

Tecnologie del linguaggio naturale

Convertitore Lessicale Italiano $\rightarrow$ Yoda

algoritmo CKY e trasformazione dell'ordine delle parole

Franco Andrea

Iudice Samuele

Olivero Alessandro

Università degli Studi di Torino

Corso di Laurea Magistrale in Informatica

Anno Accademico 2025/2026

Sommario

L'obiettivo del progetto è l'implementazione di un sistema per la conversione automatica di frasi dall'italiano allo Yodish, variante sintattica ispirata al linguaggio di Yoda nella saga di Guerre Stellari. Sviluppato in Python, il software integra un parser CKY per l'analisi a costituenti e un modulo di trasformazione ricorsiva degli alberi di derivazione. Questa relazione analizza l'intero workflow, dalla modellazione della grammatica alla manipolazione sintattica, discutendo i risultati ottenuti e i limiti dell'approccio rule-based. Nel corso della realizzazione progettuale abbiamo adottato il materiale didattico del docente come riferimento per l'apparato teorico e nozionistico, avvalendoci invece del supporto dell'intelligenza artificiale generativa come ausilio operativo nella fase di esecuzione pratica del progetto.

Indice

1	Introduzione	4
2	Background Teorico	4
2.1	Grammatiche Context-Free (CFG)	4
2.2	Forma Normale di Chomsky (CNF)	5
2.3	L'Algoritmo CKY	5
3	Struttura del Progetto	6
3.1	Il Lessico (<code>Lexicon.json</code>)	6
3.2	Le Regole Grammaticali (<code>regole_binarie.json</code>)	6
4	Implementazione	7
4.1	La Classe <code>Nodo</code>	7
4.2	L'Algoritmo CKY: Implementazione	8
4.3	La Trasformazione in Ordine Yoda	9
4.4	Linearizzazione e Post-processing	9
5	Interfaccia da Linea di Comando (CLI)	10
6	Esempio di Esecuzione	10
6.1	Albero di Derivazione Originale	10
6.2	Albero Trasformato in Ordine Yoda	11
7	Limiti e Possibili Estensioni	11
7.1	Limiti del Sistema Attuale	11
7.2	Possibili Estensioni	12
8	Confronto con Approcci Alternativi	12
8.1	Parser Rule-Based vs. Neurali	12
8.2	Trasformazione via Regole vs. Modelli Seq2Seq	13
9	Conclusioni	13

1 Introduzione

Il progetto svolto presenta un sistema di conversione sintattica dall'italiano standard (SVX) all'ordine (XSV) tipico dello yodish. Sviluppato in Python, il software modella il linguaggio tramite grammatiche context-free (CFG) in Forma Normale di Chomsky (CNF).

Lo sviluppo progettuale si articola in tre fasi principali:

- **Parsing CKY**: analisi bottom-up della frase tramite un lessico e regole grammaticali definiti in formato JSON.
- **Trasformazione**: manipolazione ricorsiva dell'albero di derivazione per il riposizionamento dei costituenti sintattici.
- **Linearizzazione**: generazione della stringa finale a partire dalle foglie dell'albero trasformato.

L'approccio adottato garantisce una netta separazione tra logica algoritmica e base di conoscenza linguistica, consentendo maggiore flessibilità e portabilità del sistema.

2 Applicazioni teoriche

2.1 Grammatiche Context-Free

Una grammatica context-free è una quadrupla $G = (V, \Sigma, R, S)$ dove:

- V è un insieme finito di simboli non-terminali.
- Σ è un insieme finito di terminali.
- R è un insieme finito di regole di produzione della forma $A \rightarrow \alpha$.
- $S \in V$ è il simbolo iniziale detto assioma.

Una stringa $w \in \Sigma^*$ è generata da G se esiste una derivazione $S \Rightarrow^* w$. Il linguaggio $L(G)$ generato dalla grammatica è l'insieme di tutte le stringhe derivabili dall'assioma.

2.2 Forma Normale di Chomsky

Un algoritmo di parsing efficiente come CKY richiede che la grammatica sia in **Forma Normale di Chomsky** (CNF). Una CFG è in CNF se tutte le regole di produzione sono della forma:

$$A \rightarrow BC \quad \text{oppure} \quad A \rightarrow a$$

dove $A, B, C \in V$ e $a \in \Sigma$, ovvero, ogni regola produce esattamente due non-terminali oppure un singolo terminale.

2.3 L'Algoritmo CKY

L'algoritmo **CKY** (da Cocke, Kasami e Younger) è un algoritmo di parsing bottom-up per grammatiche in CNF, basato su programmazione dinamica. Dato l'input $w = w_1 w_2 \dots w_n$, l'algoritmo costruisce una tabella triangolare T di dimensione $n \times n$, dove la cella $T[i][j]$ contiene l'insieme dei non-terminali che possono derivare la sottostringa $w_i w_{i+1} \dots w_j$.

Algoritmo:

1. **Inizializzazione:** per ogni i da 1 a n , inserire in $T[i][i]$ tutti i non-terminali A tali che $A \rightarrow w_i$ è una regola lessicale.
2. **Riempimento:** per ogni lunghezza di span ℓ da 2 a n , per ogni inizio i e fine $j = i + \ell - 1$, per ogni punto di taglio k da i a $j - 1$: se $B \in T[i][k]$ e $C \in T[k + 1][j]$ e $A \rightarrow BC \in R$, allora inserire A in $T[i][j]$.
3. **Accettazione:** la frase è riconosciuta se e solo se $S \in T[1][n]$.

L'efficienza dell'algoritmo deriva dalla tecnica della programmazione "dinamica", che evita di ricalcolare più volte gli stessi sotto-alberi sintattici, risolvendo il problema del parsing in tempo polinomiale invece che esponenziale.

3 Struttura Progettuale

Il progetto è organizzato in tre componenti principali:

File	Tipo	Descrizione
<code>convertitore_yoda.py</code>	Codice Python	Modulo principale: parsing, trasformazione, CLI
<code>Lexicon.json</code>	Dati JSON	Lessico italiano con etichette POS
<code>regole_binarie.json</code>	Dati JSON	Regole binarie della grammatica in CNF

3.1 Lexicon (Lexicon.json)

Lexicon è un dizionario in file JSON che mappa ogni parola italiana a una lista di categorie grammaticali (POS tag). Ad esempio:

```
"il": ["Det"], "la": ["Det"], "lo": ["Det"],
"i": ["Det"], "le": ["Det"], "gli": ["Det"], "l'": ["Det"],
"un": ["Det"], "una": ["Det"], "uno": ["Det"], "un'": ["Det"],
```

Figura 1: Organizzazione dei dati nel file Lexicon.JSON

3.2 Le Regole Grammaticali (`regole_binarie.json`)

Il file contiene le regole binarie della grammatica in CNF, dove la chiave è la coppia di figli e il valore è la lista dei possibili genitori. La scelta di invertire la direzione rispetto alla notazione tradizionale (dai figli al genitore) è una ottimizzazione utile all'algoritmo CKY: durante il parsing si cerca quali genitori sono compatibili con una coppia di figli già riconosciuti, non il contrario.

```
"NP,VP": ["S"],
"Det,N": ["NP"],
"Num,N": ["NP"],
"NP,PP": ["NP"],
"V,NP": ["VP", "VP1"],
"V,PP": ["VP"],
"V,Adj": ["VP"],
"VP1,Adv": ["VP"],
"Prep,NP": ["PP"]
```

Figura 2: Organizzazione delle regole binarie nel file JSON

La funzione `carica_dizionari()` in fase di inizializzazione converte le chiavi stringa (es. `"Det,N"`) nelle tuple Python richieste dall'algoritmo (es. `('Det', 'N')`).

4 Implementazione

4.1 La Classe Nodo

L'albero di derivazione è rappresentato tramite la classe `Nodo`, che implementa un nodo di albero n-ario con tre attributi:

- `label`: il simbolo grammaticale del nodo (es. `'NP'`, `'VP'`) oppure la parola stessa nelle foglie.
- `children`: lista di nodi figli.
- `word`: la parola (solo per le foglie lessicali).

I nodi foglia sono distinti da quelli interni tramite il metodo `is_leaf()`, che verifica se la lista `children` è vuota.

La struttura è volutamente semplice per facilitare la visita ricorsiva sia nella stampa (`pretty_print`) che nella trasformazione Yoda.

```

class Nodo:

    def __init__(self, label, children=None, word=None):
        self.label = label
        self.children = children or []
        self.word = word

    def is_leaf(self):
        return len(self.children) == 0

    def __repr__(self):
        if self.is_leaf():
            return f"[{self.label}: '{self.word}']"
        figli = ', '.join(repr(c) for c in self.children)
        return f"[{self.label} → {figli}]"

```

Figura 3: Implementazione dell'albero di derivazione

Il metodo `pretty_print()` produce una rappresentazione visuale ad albero nel terminale, con connettori grafici ASCII (`|`, `└`, `├`) per dare in output un risultato più chiaro all'utente.

4.2 L'Algoritmo CKY: Implementazione

La funzione `cky_parse(sentence)` riceve una lista di parole tokenizzate e restituisce il nodo radice dell'albero di derivazione (etichettato 'S') oppure `None` se la frase non è riconosciuta dalla grammatica.

L'implementazione usa una tabella tridimensionale `table[i][j]` dove ogni cella è un dizionario `{simbolo: Nodo}`. Questa scelta — memorizzare direttamente il nodo invece del semplice booleano — consente di ricostruire l'albero durante il parsing senza una fase separata di *backtracking*.

```

for span in range(2, n + 1):
    for i in range(n - span + 1):
        j = i + span - 1
        for k in range(i, j):
            for (B, C), parents in binary_rules.items():
                if B in table[i][k] and C in table[k+1][j]:
                    for A in parents:
                        if A not in table[i][j]:
                            node_B = table[i][k][B]
                            node_C = table[k+1][j][C]
                            table[i][j][A] = Nodo(
                                label=A,
                                children=[node_B, node_C]
                            )

```

Figura 4: Implementazione dell'algoritmo cky

L'algoritmo gestisce l'ambiguità grammaticale in modo deterministico restituendo unicamente il primo albero derivato, la cui struttura dipende esclusivamente dall'ordine di iterazione delle regole nel dizionario.

4.3 Trasformazione in Ordine Yoda

La funzione `trasforma_in_yoda()`, che visita l'albero in post-ordine (prima i figli, poi il nodo corrente) e riorganizza i sottoalberi laddove trova la struttura sintattica canonica italiana.

La regola principale è: ogni nodo S con struttura $S \rightarrow NP VP$ viene trasformato in $S_yoda \rightarrow Comp NP V$, dove **Comp** è il complemento contenuto nel VP. In altre parole, il complemento viene spostato in posizione iniziale di frase.

Sono gestiti tre casi distinti:

Caso 1: Frase transitiva semplice $VP \rightarrow V X$ con $X \in \{NP, PP, Adj\}$

$$S \rightarrow NP (VP \rightarrow V Comp) \implies S_yoda \rightarrow Comp NP V$$

Caso 2: Frase con avverbio $VP \rightarrow VP1 Adv$ con $VP1 \rightarrow V NP$

$$S \rightarrow NP (VP \rightarrow (VP1 \rightarrow V NP) Adv) \implies S_yoda \rightarrow NP Sogg V Adv$$

Caso 3: Frase intransitiva $VP \rightarrow V$ (solo verbo)

$$S \rightarrow NP (VP \rightarrow V) \implies S_yoda \rightarrow NP V$$

(nessuna trasposizione, l'ordine resta invariato poiché non c'è complemento da anteporre)

Trasformazione Caso 1 – VP con complemento diretto

```
# Caso 1: VP → V NP/PP/Adj
if child0.label == 'V' and child1.label in ('NP', 'PP', 'Adj'):
    verbo = child0
    compl = child1
    nodo.label = 'S_yoda'
    nodo.children = [compl, sogg, verbo]
```

Figura 5: Trasformazione Caso 1 – VP con complemento diretto

4.4 Linearizzazione e Post-processing

La funzione `raccogli_foglie()` effettua una visita DFS (depth-first, in-order) sull'albero trasformato, raccogliendo le parole nelle foglie da sinistra a destra. La funzione `converti_in_yoda()` applica infine un post-processing minimo:

- Le parole vengono rese minuscole, salvo i *nomi propri*, che vengono identificati come parole che appaiono nel lessico con entrambi i tag N e NP e iniziano con la maiuscola.
- La prima parola della frase Yoda viene capitalizzata.

5 Esempio di Esecuzione

Per illustrare concretamente il funzionamento del sistema, si considera la frase di input “*Tu hai amici lì*”.

5.1 Albero di Derivazione Originale

La frase viene tokenizzata in [Tu, hai, amici, lì]. Il parser CKY riconosce la struttura seguente:

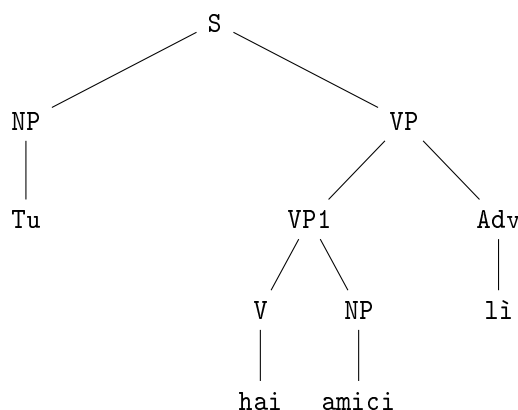


Figura 6: Albero di derivazione della frase “*Tu hai amici lì*”

5.2 Albero Trasformato in Ordine Yoda

La funzione `trasforma_in_yoda()` identifica la struttura

$S \rightarrow NP (VP \rightarrow (VP1 \rightarrow V NP) Adv)$ (Caso 2) e riorganizza le costituenti nell’ordine *Comp Sogg V Adv*:

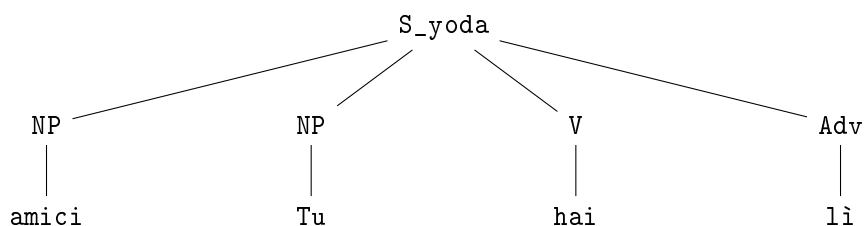


Figura 7: Albero trasformato in ordine Yoda: “*Amici tu hai lì*”

La linearizzazione delle foglie produce la frase finale:

Input ITALIANO: *Tu hai amici lì*

Output YODISH: *Amici tu hai lì*

6 Test e Limiti del Sistema

In questa sezione vengono analizzate le prestazioni del convertitore attraverso casi di test reali, verificando sia la corretta capacità di trasformazione sintattica sia le restrizioni strutturali del parser CKY implementato.

6.1 Casi di Test (Successo)

Le frasi riportate di seguito sono state elaborate correttamente, poiché i lemmi appartengono al `Lexicon.json` e le strutture rispettano le regole in Forma Normale di Chomsky (CNF) definite nel progetto.

Tipologia	Input Italiano	Output Yodish	Analisi
Semplice	<i>Luke usa la forza</i>	La forza Luke usa	Il complemento oggetto "la forza" viene correttamente anteposto al soggetto.
Con Numerale	<i>Vader distrugge due droni</i>	Due droni Vader distrugge	Il sintagma nominale (NP) composto da numerale e nome viene spostato integralmente.
Con Avverbio	<i>Io vedo sith qui</i>	Sith io vedo qui	L'avverbio "qui" mantiene la posizione finale secondo la logica del Caso 2.
Sogg. Pronome	<i>Lui mangia la mela</i>	La mela lui mangia	Corretto riconoscimento dei pronomi personali come sintagmi nominali.

Tabella 1: Esempi di trasformazioni eseguite con successo dal sistema.

6.2 Limiti del Sistema

Attraverso una fase di "stress test" con parole estratte esclusivamente dal lessico fornito, sono emersi i limiti intrinseci dell'approccio rule-based del sistema:

- **Posizionamento degli Aggettivi:** Sebbene termini come "*oscuro*" o "*potente*" siano presenti nel lessico come `Adj`, la grammatica attuale fallisce il parsing di frasi come "*Il lato oscuro vince*". Ciò è dovuto alla mancanza di una regola binaria che gestisca la sequenza `N`, `Adj` all'interno di un `NP`.
- **Ambiguità Lessicale:** Parole come "*rosso*" o "*nero*" sono caricate con doppio tag (`Adj` e `N`). Il parser restituisce solo il primo albero derivato trovato, ignorando potenziali interpretazioni alternative che potrebbero influenzare la qualità della traduzione.
- **Mancanza di Flessione (Lemmatizzazione):** Il sistema fallisce se viene inserita una forma verbale non esplicitamente mappata. Ad esempio, mentre "*Noi abbiamo*

speranza" viene riconosciuto, *"Noi avevamo speranza"* produce un errore poiché *"avevamo"* non figura nel `Lexicon.json`.

- **Strutture Complesse:** Il sistema non è in grado di processare complementi introdotti da preposizioni articolate (es. *"della"*, *"nella"*) a meno che non vengano inserite manualmente come token singoli, limitando di fatto il linguaggio naturale arbitrario.

6.3 Conclusioni sui Test

I test confermano che il cuore algoritmico (CKY e trasformazione ricorsiva) è solido e performante per l'uso didattico previsto. Tuttavia, la "Yodificazione" è attualmente limitata alla struttura della frase principale: il sistema è ottimizzato per spostare complementi diretti, ma richiede un arricchimento delle regole binarie per gestire correttamente subordinate o strutture aggettivali complesse.

7 Conclusioni e sviluppi futuri

Il progetto IT-YO ha consentito di mettere in pratica i concetti fondamentali di analisi sintattica formale appresi durante le lezioni frontali. Il sistema è funzionante per un sottoinsieme controllato di frasi italiane e produce output corretti sui casi di test previsti. La separazione tra grammatica e logica di parsing rende il sistema facilmente estendibile per coprire nuove costruzioni sintattiche è sufficiente aggiungere voci al lessico e regole al file JSON, senza modificare l'algoritmo. Dal punto di vista didattico, il progetto illustra concretamente come la teoria delle grammatiche formali si traduca in un sistema computazionale funzionante, e come la manipolazione degli alberi di derivazione sia uno strumento potente per operare trasformazioni linguistiche strutturate. Tra i possibili sviluppi futuri, l'integrazione con un PoS tagger automatico e l'estensione della grammatica a costruzioni più complesse rappresentano le direzioni più promettenti per avvicinare il sistema a un uso su frasi naturali non vincolate.